Course Title: Cryptography and Network Security Lab	**Course Code**: BCSE309P
Faculty: Prof. Karthika V	**Slot**: L43+L44
Regno: 23BCE1145	**Name**: S Vishwajith

Ex1b - Playfair and Hill Cipher

Playfair Cipher:

Code:

Sender:

```
import socket


def create_diagram(pt):
    diagram = []
    i = 0
    while i < len(pt):
        a = pt[i]
        if i + 1 < len(pt):
            b = pt[i + 1]
            if a == b:
                diagram.append(a + 'x')
                i += 1
            else:
                diagram.append(a + b)
                i += 2
        else:
            diagram.append(a + 'x')
            i += 1
    return diagram
```

```
def create_key_matrix(key):
    key = key.replace("j", "i")
    key_matrix = []
    used_chars = set()
    row = 0
    col = 0
    temp = []
    for char in key:
        if char not in used_chars and char.isalpha():
            used_chars.add(char)
            temp.append(char)
            col += 1
            if col == 5:
                key_matrix.append(temp)
                temp = []
                row += 1
                col = 0
    for i in range(ord('a'), ord('z') + 1):
        char = chr(i)
        if char == 'j':
            continue
        if char not in used_chars:
            used_chars.add(char)
            temp.append(char)
            col += 1
            if col == 5:
                key_matrix.append(temp)
                temp = []
                row += 1
                col = 0
    return key_matrix
```

```
def get_cipher(diagram, key_matrix):
    cipher_text = ""
    for pair in diagram:
        row1, col1, row2, col2 = -1, -1, -1, -1
        for r in range(5):
            for c in range(5):
                if key_matrix[r][c] == pair[0]:
                    row1, col1 = r, c
                if key_matrix[r][c] == pair[1]:
                    row2, col2 = r, c
        if row1 == row2:
            cipher_text += key_matrix[row1][(col1 + 1) % 5]
            cipher_text += key_matrix[row2][(col2 + 1) % 5]
        elif col1 == col2:
            cipher_text += key_matrix[(row1 + 1) % 5][col1]
            cipher_text += key_matrix[(row2 + 1) % 5][col2]
        else:
            cipher_text += key_matrix[row1][col2]
            cipher_text += key_matrix[row2][col1]
    return cipher_text

print("Playfair Cipher Sender initialized.")
pt = input("Enter the plaintext: ").lower()
if(len(pt) == 0):
    print("No plaintext provided. Exiting.")
    exit()
key = input("Enter the key: ").lower()
if(len(key) == 0):
    print("No key provided. Exiting.")
    exit()
```

```
diagram = create_diagram(pt)
diagram_text = ''.join(diagram)
print("Diagram created:", diagram_text)
key_matrix = create_key_matrix(key)
print("Key matrix created:")
for row in key_matrix:
    print(row)
cipher_text = get_cipher(diagram, key_matrix)
print("Ciphertext generated:", cipher_text)

s = socket.socket()
s.connect(('127.0.0.1', 5067))
s.send(cipher_text.encode().ljust(1024, b'\0'))
s.send(key.encode().ljust(1024, b'\0'))
print("Ciphertext and key sent to the receiver.")
s.close()
```

Receiver:

```
import socket

def create_key_matrix(key):
    key = key.replace("j", "i")
    key_matrix = []
    used_chars = set()
    row = 0
    col = 0
    temp = []
    for char in key:
        if char not in used_chars and char.isalpha():
            used_chars.add(char)
            temp.append(char)
```

```
            col += 1
            if col == 5:
                key_matrix.append(temp)
                temp = []
                row += 1
                col = 0
    for i in range(ord('a'), ord('z') + 1):
        char = chr(i)
        if char == 'j':
            continue
        if char not in used_chars:
            used_chars.add(char)
            temp.append(char)
            col += 1
            if col == 5:
                key_matrix.append(temp)
                temp = []
                row += 1
                col = 0
    return key_matrix

def get_plain_text(cipher, key_matrix):
    plain_text = ""
    for i in range(0, len(cipher), 2):
        pair = cipher[i:i+2]
        row1, col1, row2, col2 = -1, -1, -1, -1
        for r in range(5):
            for c in range(5):
                if key_matrix[r][c] == pair[0]:
                    row1, col1 = r, c
                if key_matrix[r][c] == pair[1]:
```

```
                row2, col2 = r, c
        if row1 == row2:
            plain_text += key_matrix[row1][(col1 - 1) % 5]
            plain_text += key_matrix[row2][(col2 - 1) % 5]
        elif col1 == col2:
            plain_text += key_matrix[(row1 - 1) % 5][col1]
            plain_text += key_matrix[(row2 - 1) % 5][col2]
        else:
            plain_text += key_matrix[row1][col2]
            plain_text += key_matrix[row2][col1]
    return plain_text

server = socket.socket()
server.bind(('127.0.0.1', 5067))
server.listen(1)
print("Playfair Cipher Receiver initialized and listening for sender.")
conn, addr = server.accept()
print(f"Connection established with {addr}.")

cipher = conn.recv(1024).decode().rstrip('\0')
key = conn.recv(1024).decode().rstrip('\0')
print("Ciphertext received:", cipher)
print("Key received:", key)
key_matrix = create_key_matrix(key)
print("Plain text reconstructed:", get_plain_text(cipher, key_matrix))

conn.close()
server.close()
```

Output:

Sender:

```
PS C:\Users\vishw\Coding\Cryptography-Sem-6\Week-2\Playfair Cipher> python .\sender.py
Playfair Cipher Sender initialized.
Enter the plaintext: hello
Enter the key: bye
Diagram created: helxlo
Key matrix created:
['b', 'y', 'e', 'a', 'c']
['d', 'f', 'g', 'h', 'i']
['k', 'l', 'm', 'n', 'o']
['p', 'q', 'r', 's', 't']
['u', 'v', 'w', 'x', 'z']
Ciphertext generated: ganvmk
Ciphertext and key sent to the receiver.
PS C:\Users\vishw\Coding\Cryptography-Sem-6\Week-2\Playfair Cipher>
```

Receiver:

```
PS C:\Users\vishw\Coding\Cryptography-Sem-6\Week-2\Playfair Cipher> python .\receiver.py
Playfair Cipher Receiver initialized and listening for sender.
Connection established with ('127.0.0.1', 57069).
Ciphertext received: ganvmk
Key received: bye
Plain text reconstructed: helxlo
PS C:\Users\vishw\Coding\Cryptography-Sem-6\Week-2\Playfair Cipher>
```

Hill Cipher:

Code:

Sender:

```
import socket

import numpy as np


def create_key_matrix(key_char_matrix):

    key_matrix = [[ord(char) - ord('a') for char in row] for row in key_char_matrix]

    return np.matrix(key_matrix, dtype=int)


def create_pt_matrix(pt, n):

    pt_matrix = []

    for i in range(0, len(pt), n):

        row = [ord(char) - ord('a') for char in pt[i:i+n]]

        pt_matrix.append(row)

    return np.matrix(pt_matrix, dtype=int)


def get_cipher_text(ct_matrix):

    ct = ''

    for row in ct_matrix:

        for num in row.tolist()[0]:

            ct += chr(num + ord('a'))

    return ct


def get_key_text(key_char_matrix):

    key = ''

    for row in key_char_matrix:

        for char in row:

            key += char

    return key
```

```
print("Hill Cipher Sender initialized.")

n = int(input("Enter the size of the key matrix (n x n): "))

pt = input(f"Enter the plaintext: ").lower()

key_char_matrix = [list(input(f"Enter row {i+1} of the key matrix (space-separated integers and length must be {n}): ").lower()) for i in range(n)]

key_matrix = create_key_matrix(key_char_matrix)

key_text = get_key_text(key_char_matrix)

print("Key Matrix:")

for row in key_matrix:

    print(row)

print(f"Key Text: {key_text}")


pt_matrix = create_pt_matrix(pt, n)

print("Plaintext Matrix:")

for row in pt_matrix:

    print(row)


ct_matrix = (pt_matrix @ key_matrix) % 26

print("Ciphertext Matrix:")

for row in ct_matrix:

    print(row)


ct = get_cipher_text(ct_matrix)

print(f"Ciphertext: {ct}")


s = socket.socket()

s.connect(('127.0.0.1', 5069))

s.send(ct.encode().ljust(1024, b'\0'))

s.send(str(n).encode().ljust(1024, b'\0'))

s.send(key_text.encode().ljust(1024, b'\0'))

print("Ciphertext, key size, and key text sent to receiver.")

s.close()
```

Receiver:

```
import socket

import numpy as np


def create_key_matrix(key_text, n):

    key_matrix = []

    index = 0

    for i in range(n):

        row = []

        for j in range(n):

            row.append(ord(key_text[index]) - ord('a'))

            index += 1

        key_matrix.append(row)

    return np.matrix(key_matrix, dtype=int)


def create_ct_matrix(ct, n):

    ct_matrix = []

    for i in range(0, len(ct), n):

        row = [ord(char) - ord('a') for char in ct[i:i+n]]

        ct_matrix.append(row)

    return np.matrix(ct_matrix, dtype=int)


def get_key_matrix_adjugate(key_matrix):

    n = key_matrix.shape[0]

    adjugate = np.zeros((n, n), dtype=int)

    for i in range(n):

        for j in range(n):

            minor = np.delete(np.delete(key_matrix, i, axis=0), j, axis=1)

            cofactor = int(round(np.linalg.det(minor))) * ((-1) ** (i + j))

            adjugate[j][i] = cofactor % 26

    return np.asmatrix(adjugate)
```

```
def get_plain_text(pt_matrix):
    pt = ''
    for row in pt_matrix:
        for num in row.tolist()[0]:
            pt += chr(num + ord('a'))
    return pt

server = socket.socket()
server.bind(('127.0.0.1', 5069))
server.listen(1)
print("Hill Cipher Receiver initialized and listening for sender.")

conn, addr = server.accept()
print(f"Connection established with sender at {addr}.")
ct = conn.recv(1024).decode().rstrip('\0')
n = int(conn.recv(1024).decode().rstrip('\0'))
key_text = conn.recv(1024).decode().rstrip('\0')

print(f"Received Ciphertext: {ct}")
print(f"Received Key Size: {n}")
print(f"Received Key Text: {key_text}")

key_matrix = create_key_matrix(key_text, n)
print("Key Matrix:")
for row in key_matrix:
    print(row)

ct_matrix = create_ct_matrix(ct, n)
print("Ciphertext Matrix:")
for row in ct_matrix:
```

```
    print(row)

det = int(round(np.linalg.det(key_matrix))) % 26
det_inverse = pow(det, -1, 26)
print(f"Determinant: {det}")
print(f"Determinant Inverse: {det_inverse}")

key_matrix_inverse = (det_inverse * get_key_matrix_adjugate(key_matrix)) % 26
print("Inverse Key Matrix:")
for row in key_matrix_inverse:
    print(row)

pt_matrix = (ct_matrix @ key_matrix_inverse) % 26
print("Plaintext Matrix:")
for row in pt_matrix:
    print(row)

pt = get_plain_text(pt_matrix)
print(f"Decrypted Plaintext: {pt}")

conn.close()
server.close()
```

Output:

Sender:

```
PS C:\Users\vishw\Coding\Cryptography-Sem-6\Week-2\Hill Cipher> python .\sender.py
Hill Cipher Sender initialized.
Enter the size of the key matrix (n x n): 3
Enter the plaintext: paymoremoney
Enter row 1 of the key matrix (space-separated integers and length must be 3): rrf
Enter row 2 of the key matrix (space-separated integers and length must be 3): vsv
Enter row 3 of the key matrix (space-separated integers and length must be 3): cct
Key Matrix:
[[17 17  5]]
[[21 18 21]]
[[ 2  2 19]]
Key Text: rrfvsvcct
Plaintext Matrix:
[[15  0 24]]
[[12 14 17]]
[[ 4 12 14]]
[[13  4 24]]
Ciphertext Matrix:
[[17 17 11]]
[[12 22  1]]
[[10  0 18]]
[[15  3  7]]
Ciphertext: rrlmwbkaspdh
Ciphertext, key size, and key text sent to receiver.
PS C:\Users\vishw\Coding\Cryptography-Sem-6\Week-2\Hill Cipher>
```

Receiver:

```
PS C:\Users\vishw\Coding\Cryptography-Sem-6\Week-2\Hill Cipher> python .\receiver.py
Hill Cipher Receiver initialized and listening for sender.
Connection established with sender at ('127.0.0.1', 49701).
Received Ciphertext: rrlmwbkaspdh
Received Key Size: 3
Received Key Text: rrfvsvcct
Key Matrix:
[[17 17  5]]
[[21 18 21]]
[[ 2  2 19]]
Ciphertext Matrix:
[[17 17 11]]
[[12 22  1]]
[[10  0 18]]
[[15  3  7]]
Determinant: 23
Determinant Inverse: 17
Inverse Key Matrix:
[[ 4  9 15]]
[[15 17  6]]
[[24  0 17]]
Plaintext Matrix:
[[15  0 24]]
[[12 14 17]]
[[ 4 12 14]]
[[13  4 24]]
Decrypted Plaintext: paymoremoney
PS C:\Users\vishw\Coding\Cryptography-Sem-6\Week-2\Hill Cipher>
```